

Máster en Desarrollo Full Stack

Proyecto Final de Máster

Proyecto Final del Máster en Desarrollo Full Stack	2
Objetivos generales	2
Estructura del proyecto	2
1. Definición del problema	2
2. Reflexión: aportación y eficiencia	2
3. Listado de tecnologías utilizadas	2
4. Definición de tipos de usuarios	2
5. Casos de uso	3
6. Seguridad y protección de datos	3
Requisitos técnicos mínimos	3
Procedimiento de entrega	3
1. Requisitos previos	3
2. Material obligatorio a entregar	3
3. Proceso de revisión	5
4. Defensa del proyecto	5
5. Evaluación y calificación	6
6. Reentregas y segundas convocatorias	6
7. Consejos finales del equipo académico	7
Criterios de evaluación	7
Recomendaciones del director académico	7

Proyecto Final del Máster en Desarrollo Full Stack

El Proyecto Final de Máster (PFM) representa la culminación del programa formativo. Su objetivo es que el alumno demuestre haber adquirido las competencias técnicas y metodológicas necesarias para desarrollar, documentar y desplegar una aplicación web funcional, integrando tanto el backend (Django) como elementos del frontend (React u otra tecnología).

Objetivos generales

- Aplicar los conocimientos adquiridos en backend con Django y frontend con React, aunque el proyecto principal se base en Django
- Desarrollar una aplicación web completa, con flujo real de datos y despliegue operativo.
- Mostrar la capacidad de análisis, diseño, desarrollo, documentación y defensa técnica del trabajo.
- Comprender cómo las herramientas aprendidas pueden aportar valor real y eficiencia en un entorno profesional.
- Demostrar autonomía técnica y responsabilidad profesional en el ciclo de desarrollo completo.

Estructura del proyecto

1. Definición del problema

Todo proyecto debe partir de la identificación de una necesidad o problema real que pueda ser abordado mediante una aplicación web.

Debe incluir: contexto, carencias detectadas, impacto y la solución propuesta.

2. Reflexión: aportación y eficiencia

Documento en el que se analice cómo la herramienta mejora procesos, qué tareas se automatizan o simplifican, y qué beneficios aporta.

3. Listado de tecnologías utilizadas

Debe detallar las tecnologías empleadas y justificar su elección:

- Backend (obligatorio): Django, base de datos, librerías, autenticación.
- Frontend (parcialmente obligatorio): Al menos una o dos vistas con React que obtengan datos reales del backend.
- DevOps y despliegue: Git, GitHub, VPS o nube, Docker (opcional).

4. Definición de tipos de usuarios

Describir los roles existentes en la aplicación y las acciones permitidas para cada uno (Administrador, Usuario registrado, Visitante, etc.).

5. Casos de uso

Explicar varios casos de uso representativos por tipo de usuario, indicando flujo, resultado y posibles errores.

6. Seguridad y protección de datos

Analizar las medidas de seguridad aplicadas: autenticación, validación, protección ante CSRF/XSS, hashing de contraseñas, gestión de secretos, HTTPS, copias de seguridad.

Requisitos técnicos mínimos

1. Backend en Django con modelos, vistas, plantillas, autenticación y persistencia de datos.
2. Frontend con integración React mínima (una o dos vistas que obtengan datos del backend).
3. Despliegue funcional en URL pública.
4. Repositorio GitHub público con README y commits significativos.

Procedimiento de entrega

1. Requisitos previos

Antes de iniciar el desarrollo del Proyecto Final de Máster (PFM), el alumno deberá haber **entregado y aprobado todas las actividades obligatorias** correspondientes a los módulos formativos del máster correspondientes a Prework, Frontend y Backend.

Solo se podrá iniciar el trabajo final una vez el equipo docente confirme que el alumno cumple estos requisitos.

El objetivo de esta medida es garantizar que el alumno dispone de los conocimientos y habilidades necesarias para abordar un proyecto completo de desarrollo full stack con autonomía y rigor.

2. Material obligatorio a entregar

La entrega deberá incluir **cuatro elementos imprescindibles**, que serán revisados conjuntamente:

1. Repositorio en GitHub (obligatorio)

- Contendrá todo el código fuente del proyecto, tanto del backend como del frontend.

- Deberá ser público o compartido con el equipo docente.
- El repositorio deberá incluir:
 - Un archivo README.md con instrucciones de instalación, configuración y uso.
 - Historial de commits claros y progresivos que muestren la evolución del trabajo.
 - Estructura limpia y coherente del código.

2. Aplicación desplegada (obligatorio)

- La aplicación deberá estar **accesible públicamente mediante una URL navegable**.
- El despliegue podrá realizarse en un VPS o servicio en la nube (ej. DigitalOcean, Render, Railway, AWS, etc.).
- No se exige dominio propio, pero sí un acceso estable y funcional.
- El alumno será responsable de mantener la aplicación en línea durante todo el proceso de revisión y defensa.

3. Documento del proyecto (obligatorio)

- Documento en formato **PDF o Word** con todos los apartados establecidos en la guía (definición del problema, reflexión, tecnologías, usuarios, casos de uso y seguridad).
- Este documento servirá como memoria técnica del proyecto y deberá incluir capturas de pantalla, diagramas y ejemplos que ilustren el funcionamiento de la herramienta.
- El estilo debe ser claro, formal y ordenado, cuidando redacción y ortografía.

4. Vídeo explicativo (obligatorio)

- Duración máxima: **5 minutos**.

- Formato libre (puede grabarse con Loom, OBS, Screencast, Zoom, etc.).
- Debe mostrar brevemente:
 - El problema o contexto.
 - Una demostración funcional de la herramienta.
 - Una mención a las tecnologías utilizadas.
 - Ejemplo del uso de React para obtener o mostrar datos desde el backend.
- El vídeo deberá estar accesible mediante un enlace (YouTube, Drive, Loom o similar).

3. Proceso de revisión

- Una vez entregado todo el material, el **equipo docente realizará una revisión técnica y documental** del proyecto.
- En esta fase se comprobará:
 - Que todos los elementos exigidos se han entregado correctamente.
 - Que el proyecto cumple los requisitos mínimos de funcionalidad y coherencia técnica.
 - Que el código y la documentación sean originales y elaborados por el propio alumno.
- Si el proyecto cumple con los requisitos, se enviará al alumno un **enlace de Calendly** para agendar la **defensa del proyecto**.

4. Defensa del proyecto

- La defensa se realizará mediante **videollamada de una duración máxima de 20 minutos**.
- Participarán el alumno y **uno o varios profesores del máster**.

- Durante la defensa, el equipo docente podrá:
 - Solicitar al alumno que explique decisiones de diseño, arquitectura o seguridad.
 - Pedir que muestre partes concretas del código.
 - Preguntar sobre las tecnologías utilizadas, la lógica de negocio o la estructura del proyecto.
 - Evaluar si la herramienta se ajusta al problema planteado inicialmente.
- El objetivo de la defensa no es únicamente comprobar el funcionamiento del proyecto, sino **validar la autoría y comprensión técnica integral** del alumno.

5. Evaluación y calificación

Una vez completada la defensa, el equipo docente:

- Emitirá una **evaluación técnica y cualitativa**, considerando los criterios establecidos en la rúbrica oficial.
- Comunicará la nota final y el feedback correspondiente en un plazo máximo de **10 días hábiles** tras la defensa.
- En caso de necesitar correcciones menores, se podrán solicitar ajustes sin requerir una nueva defensa.

6. Reentregas y segundas convocatorias

- Cada alumno dispone de **una única oportunidad de defensa por trimestre académico**.
- Si el proyecto no es superado, el alumno deberá esperar al siguiente trimestre para volver a presentarlo.
- En la reentrega, deberá incluir **una versión revisada y mejorada** del proyecto, atendiendo las observaciones del equipo docente.

- No se permite presentar el mismo proyecto en convocatorias consecutivas sin mejoras significativas.

7. Consejos finales del equipo académico

- Revisa cuidadosamente los criterios de evaluación antes de entregar.
- Comprueba que la aplicación está desplegada y accesible desde cualquier navegador.
- Practica una **defensa oral clara, concisa y segura**, priorizando comprensión sobre complejidad.
- Recuerda que **la calidad, la claridad y la coherencia del trabajo pesan más que la cantidad de funcionalidades**.

Criterios de evaluación

Criterio	Peso
Correctitud técnica del backend (Django)	30%
Integración parcial de React (consumo de datos)	15%
Arquitectura, estructura y buenas prácticas	15%
Seguridad e integridad de datos	15%
Documentación y presentación	15%
Reflexión y valor aportado	10%

Nota mínima de aprobación: 7.0 / 10

Recomendaciones del director académico

- Define el problema y los usuarios antes de escribir código.
- Prioriza un MVP funcional y coherente.
- Asegura que la integración con React funcione correctamente.
- Documenta las decisiones técnicas tomadas.
- Prepárate para explicar cada parte del proyecto durante la defensa.